

GUI

Graphical User Interface

INTRODUCTION:

GUI stands for **Graphical User Interface**. It's a user interface that allows users to interact with electronic devices using graphical icons, buttons, and visual indicators rather than text-based commands. GUIs are commonly used in modern operating systems (like Windows, macOS, and Linux distributions) and applications to make it easier for users to navigate and interact with software intuitively.

In a GUI:

- **Icons** represent files, applications, and actions.
- **Buttons** perform specific actions when clicked.
- **Menus** organize commands and functions for easy access.
- **Windows** displays content, applications, or dialogs that users can interact with.

The main benefit of a GUI is that it makes software accessible to a wide range of users, even those who aren't familiar with command-line interfaces or programming.

Command Line Interface

Invented in 1970s



The collage consists of three oval-shaped images. The left image shows a grey sign for 'parc A Xerox Company' with the address '3333 COYOTE HILL ROAD' below it. The middle image is a blue icon of two hands shaking. The right image shows two men in suits standing behind two early personal computers, likely the Xerox 8010 Star Information System.

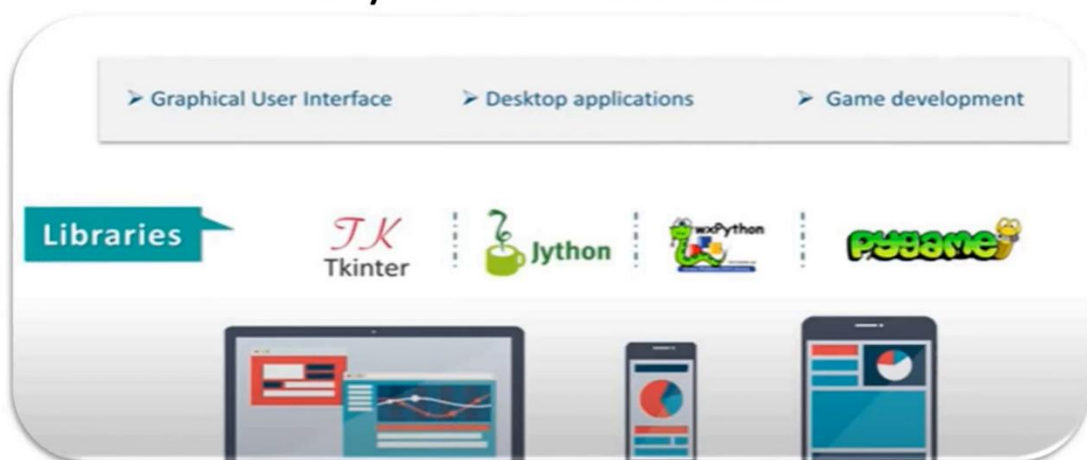
with electronic devices via graphical elements

Downloaded from <http://www.jstor.org/stable/2346122> Tue, 27 Jun 2016 12:02:00 UTC

GUIs



Python Libraries





1. Creating A GUI In Python:

Creating a GUI (Graphical User Interface) in Python is a common task that can be done using several libraries. The most popular libraries for creating GUI applications in Python are Tkinter, PyQt, Kivy, and wxPython. Below is an example of how to create a simple GUI application using Tkinter, which is included with Python by default.

Steps to Create a Simple GUI in Python with Tkinter

Install Tkinter (if not already installed): Tkinter is included with Python, so you typically don't need to install it separately. But if you're using a minimal installation, you can install it with:

```
pip install tk
```

Code to create a basic window with Tkinter:

Here's an example where we create a window with a label and a button that, when clicked, changes the label text:

```
import tkinter as tk

# Create the main window

root = tk.Tk()

root.title("Simple Tkinter GUI") # Title of the window

root.geometry("300x200") # Size of the window


# Create a label widget

label = tk.Label(root, text="Hello, Tkinter!", font=("Helvetica", 14))

label.pack(pady=20) # Add the label to the window with some padding


# Function to update the label text

def change_text():

    label.config(text="You clicked the button!")


# Create a button widget

button = tk.Button(root, text="Click Me", command=change_text,
font=("Helvetica", 12))
```

```
button.pack(pady=10) # Add the button to the window with some padding
```

```
# Run the Tkinter event loop
```

```
root.mainloop()
```

Explanation of the Code:

`tk.Tk()`: This initializes the main window. It's the root widget that will contain all other elements.

`root.title("Simple Tkinter GUI")`: This sets the title of the window.

`root.geometry("300x200")`: This sets the size of the window to 300px by 200px.

`tk.Label()`: Creates a label widget that displays text in the window.

`tk.Button()`: Creates a button widget that users can click. The command parameter specifies the function to call when the button is clicked.

`root.mainloop()`: This starts the Tkinter event loop, which listens for user actions like button clicks and updates the GUI accordingly.

Widget Classes:

In many programming frameworks, particularly in GUI (Graphical User Interface) toolkits, a **widget** is a visual component that can perform various functions, like displaying text, accepting user input, showing images, etc. Here's a general overview of common widget classes in GUI libraries like Tkinter (Python), PyQt, and web frameworks.

1. Basic Widgets

- **Label**: Displays static text or images. For example, a "Hello, World!" text or an icon.
- **Button**: Triggers an action when clicked. Buttons can include text, images, or icons.

- **Entry / Text Box:** Allows users to input single or multiline text.
- **Checkbutton / Checkbox:** A box that can be checked or unchecked to represent an option's selection.
- **Radiobutton:** Allows users to select one option from a group of options.
- **Slider:** Provides a way to select a numerical value by sliding a marker along a line.
- **ComboBox / Dropdown:** Shows a dropdown list from which a user can select an option.

2. Container Widgets

- **Frame:** Acts as a container to hold other widgets. Frames help organize the layout.
- **PanedWindow:** A split window that can hold other widgets and allows resizing between panes.
- **Tab / Notebook:** Allows different content to be organized into tabs.
- **Scroll Area / ScrolledWindow:** Provides scrolling capability to content that may be too large to display at once.

3. Display Widgets

- **Canvas:** A widget for drawing shapes, text, or custom graphics.
- **Image / Icon Widget:** Displays an image or an icon; often used in combination with other widgets.
- **Progress Bar:** Visualizes progress for tasks that take time.

4. Advanced Input Widgets

- **Spinbox:** Provides a way to choose a value by scrolling through a range (e.g., numbers).
- **TreeView / Table:** Displays data in a tabular or tree-like structure; useful for displaying structured information.
- **Date / Time Picker:** Allows users to select a date or time from a predefined widget.

5. Message / Dialog Widgets

- **MessageBox:** Shows messages, alerts, or confirmations to users.
- **File Dialog:** Opens a dialog to select files or directories.
- **Color Picker:** Allows users to select a color from a palette.

- **Custom Dialogs:** Designed for specific user interactions (e.g., a login dialog).

Examples of Widget Libraries by Language/Framework:

- **Python (Tkinter, PyQt, Kivy):** These libraries provide most of the above widgets and are commonly used for desktop applications.
- **JavaScript (HTML/CSS with libraries like React, Vue):** For web-based UIs, using components such as `<input>`, `<button>`, and libraries offering reusable components.
- **Java Swing / JavaFX:** Java has its own set of widget classes for building desktop applications.

Each GUI library has unique properties and functions for these widget classes but generally serves similar purposes across different platforms and languages.

Working with Frames in Tkinter

Frame: A container widget used to hold other widgets. It is useful for organizing your layout.

`pack()`, `grid()`, or `place()` can be used for placing frames and widgets inside frames.

`bg` sets the background color of the frame.

`bd` sets the border width of the frame.

The `fill` and `expand` options in `pack()` allow you to control how the frame behaves when the window is resized.

Layout Manager:

In Python, the LayoutManager is typically used with GUI frameworks like Tkinter to organize widgets (like buttons, labels, text fields, etc.) in a specific layout. Tkinter provides several layout managers to help position the widgets, such as `pack()`, `grid()`, and `place()`. Here's an example of how you can use the grid layout manager in Tkinter to arrange widgets:

`grid()` method is used to place widgets in a table-like grid.

`row` and `column` define the position of the widget.

`padx` and `pady` provide padding around the widget for better spacing.

`columnspan` allows a widget to span multiple columns (e.g., the second button spans two columns).

a Layout Manager:

- Use **pack** for simple stacking or alignment of widgets.
- Use **grid** for more complex, structured layouts.
- Use **place** when you need precise control over widget positioning.

Event Handling:

In Python GUI programming, event handling refers to the process of responding to user interactions with the application, such as clicking a button, typing in a text field, or hovering the mouse. In Tkinter (the standard Python GUI library), event handling is achieved by binding events to widgets, such as buttons, labels, or windows.

Code's:

```
'''
```

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
# Create a label widget with the text "Hello world!"
```

```
label = Label(root, text="Hello world!")
```

```
# Place the label widget in the window
```

```
label.pack()
```

```
# Run the main loop to keep the window open
```

```
root.mainloop()
```

```
'''
```

```
#2
```

```
'''
```

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
# Create a label widget with the text "Hello world!"
```

```
label = Label(root, text="Hello world!")
```

```
label1 = Label(root, text="Lable 2")
```

```
# Place the label widget in the window
```

```
label.grid(row=0, column=0)
```

```
# Place the label widget in the window
```

```
label1.grid(row=0, column=1)
```

```
# Run the main loop to keep the window open
```

```
root.mainloop()
```

```
'''
```

```
#3
```

```
'''
```

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
def onclick():
```

```
    label=Label(root,text="You have clicked this button")
```

```
    label.pack()
```

```
button = Button(root,text="Click this button",  
command=onclick , fg="blue", bg="green")#fg is four ground  
button.pack()
```

```
root.mainloop()
```

```
'''
```

```
#4
```

```
'''
```

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
def onclick():
```

```
    label = Label(root, text="You have clicked this button")
```

```
    label.pack()
```

```
# First button (disabled)
```

```
button = Button(root, text="Click this button", command=onclick,  
state=DISABLED)
```

```
button.pack(side=LEFT, padx=10) # Add the first button with spacing
```

```
# Second button (enabled)
```

```
button1 = Button(root, text="button", command=onclick)
```

```
button1.pack(side=LEFT, padx=10) # Add the second button with spacing
```

```
root.mainloop()
```

```
'''
```


#5.

'''

from tkinter import *

Create the main window

root = Tk()

def button1_action():

label = Label(root, text="You clicked Button 1!")

label.pack()

def button2_action():

label = Label(root, text="You clicked Button 2!")

label.pack()

First button with different text

button1 = Button(root, text="Button 1", command=button1_action)


```
button1.pack(pady=10) # Adds some vertical space

# Second button with different text
button2 = Button(root, text="Button 2", command=button2_action)
button2.pack(pady=10) # Adds some vertical space

# Run the main event loop
root.mainloop()

'''
```

#6.

'''

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
input =Entry(root, width=100)
```

```
input.pack()
```

```
def onclick():
```

```
    label=Label(root,text="You have clicked this button")
```

```
    label.pack()
```

```
button = Button(root,text="Click this button", command=onclick , fg="blue",  
bg="green")#fg is four ground
```

```
button.pack()
```

```
root.mainloop()
```

```
'''
```

```
#7
```

```
'''
```

```
#Getting Input from User
```

```
from tkinter import *
```

```
# Create the main window
```

```
root = Tk()
```

```
input =Entry(root, width=100)
```

```
input.pack()
```

```
def onclick():
```

```
    label=Label(root,text=input.get())
```

```
    label.pack()
```

```
button = Button(root,text="Click this button", command=onclick , fg="blue",  
bg="green")#fg is four ground
```

```
button.pack()
```

```
root.mainloop()
```

```
'''
```

```
#8.
```

```
'''
```

```
# Working with Frames in Tkinter
```

```
import tkinter as tk
```

```
root = tk.Tk()
```

```
root.title("Tkinter Frame with Grid Example")
```

```
# Create Frame
```

```
frame = tk.Frame(root)
```

```
frame.grid(row=0, column=0)
```

```
# Add widgets inside the frame using grid layout
```

```
label = tk.Label(frame, text="Name:")
```

```
label.grid(row=0, column=0)
```

```
entry = tk.Entry(frame)
```

```
entry.grid(row=0, column=1)
```

```
submit_button = tk.Button(frame, text="Submit")
```

```
submit_button.grid(row=1, column=0, columnspan=2)
```

```
root.mainloop()
```

```
'''
```

```
#9.
```

```
'''
```

```
import tkinter as tk
```

```
# Create the main window
```

```
root = tk.Tk()
```

```
root.title("Layout Manager Example")
```

```
# Create some labels and buttons
```

```
label1 = tk.Label(root, text="Label 1")
```

```
label2 = tk.Label(root, text="Label 2")
```

```
label3 = tk.Label(root, text="Label 3")
```

```
button1 = tk.Button(root, text="Button 1")
```

```
button2 = tk.Button(root, text="Button 2")
```

```
# Grid Layout: Define where to place each widget
```

```
label1.grid(row=0, column=0, padx=10, pady=10)
```

```
label2.grid(row=0, column=1, padx=10, pady=10)
```

```
label3.grid(row=1, column=0, padx=10, pady=10)
```

```
button1.grid(row=1, column=1, padx=10, pady=10)
```

```
button2.grid(row=2, column=0, colspan=2, pady=10)
```

```
# Run the Tkinter event loop
```

```
root.mainloop()
```

```
'''
```

#10 EVENT HANDLING

```
'''
```

```
import tkinter as tk
```

```
# Function to handle button click event
```

```
def on_button_click(event):
```

```
    label.config(text="Button clicked!")
```

```
# Function to handle key press event
```

```
def on_key_press(event):
```

```
    label.config(text=f"Key pressed: {event.char}")
```

```
# Create the main window
```

```
root = tk.Tk()
```

```
root.title("Event Handling Example")
```

```
# Create a label
```

```
label = tk.Label(root, text="Hello, click the button or press a key!")
```

```
label.pack(pady=20)
```

```
# Create a button
```

```
button = tk.Button(root, text="Click Me")
```

```
button.pack(pady=20)
```

```
# Bind the button click event
```

```
button.bind("<Button-1>", on_button_click) # Left mouse click event
```

```
# Bind a key press event to the main window
```

```
root.bind("<KeyPress>", on_key_press) # Any key press event
```

```
# Run the Tkinter event loop
```

```
root.mainloop()
```



```
'''
```

```
### 10 calculator
```

```
'''
```

```
from tkinter import *
```

```
root = Tk()
```

```
input = Entry(root, width=50)
```

```
input.grid(row=0, column=0, columnspan=3, padx=15, pady=15)
```

```
def click(num):
```

```
    current = input.get()
```

```
    input.delete(0, END)
```

```
    input.insert(0, str(current) + str(num))
```

```
    return
```

```
def add():
```

```
    current = input.get()
```

```
    global fnum
```

```
    fnum = int(current)
```

```
    input.delete(0, END)
```

```
    return
```

```
def clear():
```

```
    input.delete(0, END)
```

```
    return
```

```
def equal():
```

```
    current = input.get()
```

```
    snum = int(current)
```

```
    input.delete(0, END)
```

```
    input.insert(0, str(fnum + snum))
```

```
    return
```

```
button_1 = Button(root, text="1", padx=50, pady=25, command=lambda: click(1))
```

```
button_2 = Button(root, text="2", padx=50, pady=25, command=lambda: click(2))
```

```
button_3 = Button(root, text="3", padx=50, pady=25, command=lambda: click(3))
```

```
button_4 = Button(root, text="4", padx=50, pady=25, command=lambda: click(4))
```

```
button_5 = Button(root, text="5", padx=50, pady=25, command=lambda: click(5))
```

```
button_6 = Button(root, text="6", padx=50, pady=25, command=lambda: click(6))
```

```
button_7 = Button(root, text="7", padx=50, pady=25, command=lambda: click(7))
```

```
button_8 = Button(root, text="8", padx=50, pady=25, command=lambda: click(8))
```

```
button_9 = Button(root, text="9", padx=50, pady=25, command=lambda: click(9))
```

```
button_0 = Button(root, text="0", padx=50, pady=25, command=lambda: click(0))
```

```
button_add = Button(root, text="+", padx=105, pady=25, command=add)
```

```
button_clear = Button(root, text="AC", padx=45, pady=25, command=clear)
```

```
button_equal = Button(root, text="=", padx=105, pady=25, command=equal)
```

```
button_7.grid(row=1, column=0)
```

```
button_8.grid(row=1, column=1)
```

```
button_9.grid(row=1, column=2)
```

```
button_4.grid(row=2, column=0)
```

```
button_5.grid(row=2, column=1)
```

```
button_6.grid(row=2, column=2)
```

```
button_1.grid(row=3, column=0)
```

```
button_2.grid(row=3, column=1)
```

```
button_3.grid(row=3, column=2)
```

```
button_0.grid(row=4, column=0)
```

```
button_add.grid(row=4, column=1, colspan=2)
```

```
button_clear.grid(row=5, column=0)
```

```
button_equal.grid(row=5, column=1, colspan=2)
```

```
root.mainloop()
```

```
'''
```

```
'''
```

```
#color chooser
```

```
from tkinter.colorchooser import askcolor
```

```
from tkinter import *
```

```
top = Tk()
```

```
top.geometry("100x100")
```

```
def show():
```

```
    color = askcolor()
```

```
    print(color)
```

```
B = Button(top, text = "Click", command = show)
```

```
B.place(x=50,y=50)
```

```
top.mainloop()
```

```
'''
```

'''

from tkinter import *

from tkinter import messagebox

```
top = Tk()
C = Canvas(top, bg="blue", height=250, width=300)
coord = 10, 50, 240, 210
arc = C.create_arc(coord, start=0, extent=150, fill="red")
line = C.create_line(10,10,200,200,fill='white')
C.pack()
top.mainloop()
'''
```

```
#login form
```

```
'''
```

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
# Function to validate login
```

```
def login():
```

```
    username = username_entry.get()
```

```
    password = password_entry.get()
```

```
# Hardcoded credentials for demonstration
```

```
if username == "admin" and password == "password123":
```

```
    messagebox.showinfo("Login Successful", "Welcome to the system!")
```

```
else:
```

```
    messagebox.showerror("Login Failed", "Invalid username or password.")
```



```
# Create the main window
```

```
window = tk.Tk()
```

```
window.title("Login Form")
```

```
window.geometry("300x200")
```

```
# Username label and entry
```

```
username_label = tk.Label(window, text="Username:")
```

```
username_label.pack(pady=5)
```

```
username_entry = tk.Entry(window)
```

```
username_entry.pack(pady=5)
```

```
# Password label and entry
```

```
password_label = tk.Label(window, text="Password:")
```

```
password_label.pack(pady=5)
```

```
password_entry = tk.Entry(window, show="*")
```

```
password_entry.pack(pady=5)
```

```
# Login button
```

```
login_button = tk.Button(window, text="Login", command=login, fg="blue",  
bg="green")
```

```
login_button.pack(pady=10)
```

```
# Run the application
```

```
window.mainloop()
```

```
'''
```

'''

```
import tkinter as tk
```

```
from tkinter import messagebox
```

```
# Function to handle login button click
```

```
def login():
```

```
    username = username_entry.get()
```

```
    password = password_entry.get()
```

```
# Simple validation logic
```

```
if username == "admin" and password == "password":
```

```
    messagebox.showinfo("Login Successful", "Welcome!")
```

```
else:
```

```
    messagebox.showerror("Login Failed", "Invalid credentials, please try  
again.")
```

```
# Initialize the main window
```

```
root = tk.Tk()
```

```
root.title("Login Form")
```

```
root.geometry("400x300")
```

```
# Create a Canvas
```

```
canvas = tk.Canvas(root, bg="lightblue", height=300, width=400)
```

```
canvas.pack(fill="both", expand=True)
```

```
# Draw a rectangle for the form background
```

```
canvas.create_rectangle(50, 50, 350, 250, fill="white", outline="gray")
```

```
# Add Labels and Entries
```

```
canvas.create_text(200, 80, text="Login Form", font=("Arial", 16, "bold"),  
fill="black")
```

```
canvas.create_text(120, 120, text="Username:", font=("Arial", 12), fill="black")
```

```
username_entry = tk.Entry(root)
```

```
canvas.create_window(240, 120, window=username_entry, width=140)
```

```
canvas.create_text(120, 160, text="Password:", font=("Arial", 12), fill="black")
```

```
password_entry = tk.Entry(root, show="*")
```

```
canvas.create_window(240, 160, window=password_entry, width=140)
```

```
# Add Login Button
```

```
login_button = tk.Button(root, text="Login", command=login, bg="blue",  
fg="white")
```

```
canvas.create_window(200, 200, window=login_button, width=80, height=30)
```

```
# Run the application
```

```
root.mainloop()
```

```
'''
```